`std::thread::hardware_concurrency`. As I said already, unless you're on a massively parallel machine where threads are cheap, this is probably a bad idea, but it shows the overall structure. It would be possible to have fewer threads, with each thread handling several values from the source range, but there will come a point where there are sufficiently few threads that this is less efficient than the forward-propagation algorithm.

Anyway, the key work is done in the function call operator of `process_element`. At each step you either take the *i*th element from the original range or the *i*th element from the buffer ❷ and add it to the value `stride` elements prior ❸, storing it in the buffer if you started in the original range or back in the original range if you started in the buffer ❹. You then wait on the barrier ❺ before starting the next step. You've finished when the `stride` takes you off the start of the range, in which case you need to update the element in the original range if your final result was stored in the buffer ❻. Finally, you tell the barrier that you're `done_waiting()` ❼.

Note that this solution isn't exception safe. If an exception is thrown in `process_element` on one of the worker threads, it will terminate the application. You could deal with this by using a `std::promise` to store the exception, as you did for the `parallel_find` implementation from listing 8.9, or even just using a `std::exception_ptr` protected by a mutex.

That concludes our three examples. Hopefully, they've helped to crystallize some of the design considerations highlighted in sections 8.1, 8.2, 8.3, and 8.4 and have demonstrated how these techniques can be brought to bear in real code.

8.6. Summary

We've covered quite a lot of ground in this chapter. We started with various techniques for dividing work between threads, such as dividing the data beforehand or using a number of threads to form a pipeline. We then looked